Celery — Full Notes (Beginner to Advanced)

Level: Beginner → Intermediate → Advanced

Stack: Python, Django, Celery, Redis

Goal: Background Tasks, Asynchronous Processing, Scheduled Tasks, Production Deployment

---

1. What Is Celery?

Celery হলো Python-এর একটি distributed task queue system। এটি application-এর ভারী বা সময়সাপেক্ষ কাজ background-এ execute করতে সাহায্য করে।

উদাহরণ:

- User registration-এর পরে email পাঠানো
- Password reset email পাঠানো
- Image processing
- PDF report তৈরি করা
- Bulk notification পাঠানো
- Scheduled বা periodic task চালানো
- Data processing ও API integration

Celery ছাড়া

```
def register_user(request):
    user = create_user(request.POST)
    send_welcome_email(user.email)
    return response
```

এখানে email পাঠানো শেষ না হওয়া পর্যন্ত request আটকে থাকতে পারে।

Celery দিয়ে

```
def register_user(request):
    user = create_user(request.POST)
    send_welcome_email_task.delay(user.id)
    return response
```

এখন Django task-টি queue-তে পাঠায় এবং worker background-এ কাজটি সম্পন্ন করে।

মনে রাখবে: `.delay()` task-কে background-এ চালানোর জন্য queue-তে পাঠায়; এটি নিজে task execute করে না।

---

2. How Celery Works

Celery-এর প্রধান components:

1. **Producer** — task তৈরি করে; সাধারণত Django application।
2. **Broker** — task queue-তে রাখে; যেমন Redis বা RabbitMQ।
3. **Worker** — queue থেকে task নিয়ে execute করে।
4. **Result Backend** — task-এর result বা state সংরক্ষণ করে, যদি configure করা থাকে।
5. **Beat Scheduler** — নির্দিষ্ট সময় পরপর task schedule করে।

Architecture

```
User Request
     |
     v
Django Application
     |
     | .delay()
     v
Message Broker (Redis / RabbitMQ)
     |
     v
Celery Worker
     |
     v
Execute Task
     |
     v
Result Backend (Optional)
```

Celery এবং Redis-এর পার্থক্য

Technology	Main responsibility
Django	Web application ও API

Celery	Background task execution
Redis	Task broker, cache বা result backend
Celery Beat	Periodic task scheduling
PostgreSQL	Persistent relational data

Redis broker হিসেবে ব্যবহার করলে task message queue করার জন্য Redis ব্যবহৃত হয়। Redis result backend হিসেবেও ব্যবহার করা যায়, তবে সেটি আলাদা configuration।

---

3. Install Celery and Redis

Virtual environment activate করো:

```
python -m venv .venv
```

Windows:

```
.venv\Scripts\activate
```

Linux/macOS:

```
source .venv/bin/activate
```

Celery ও Redis Python client install করো:

```
pip install celery redis
```

Django project হলে:

```
pip install django celery redis
```

`requirements.txt` তৈরি করতে:

```
pip freeze > requirements.txt
```

Redis চালানো

Redis server চালু থাকতে হবে। Local Redis installation অথবা Docker ব্যবহার করা যায়।

Docker দিয়ে:

```
docker run -d \
  --name redis \
  -p 6379:6379 \
  redis:7
```

Windows PowerShell-এ একই command এক লাইনে চালাতে পারো:

```
docker run -d --name redis -p 6379:6379 redis:7
```

Redis connection test:

```
redis-cli ping
```

Expected output:

```
PONG
```

Docker container-এর Redis test:

```
docker exec -it redis redis-cli ping
```

Production-এ authentication, network isolation এবং persistent storage configure না করে Redis-কে public internet-এ expose করবে না।

---

4. Basic Celery Project Without Django

প্রথমে `celery_demo` নামে একটি directory তৈরি করো।

```
celery_demo/
├── celery_app.py
└── tasks.py
```

Step 1: Create `celery_app.py`

```
from celery import Celery
```

```
app = Celery(
    "celery_demo",
    broker="redis://localhost:6379/0",
    backend="redis://localhost:6379/1",
)

app.conf.update(
    task_serializer="json",
    accept_content=["json"],
    result_serializer="json",
    timezone="Asia/Dhaka",
    enable_utc=True,
)
```

এখানে:

- Redis database `0` broker হিসেবে ব্যবহৃত হচ্ছে।
- Redis database `1` result backend হিসেবে ব্যবহৃত হচ্ছে।
- JSON task data serialize করতে ব্যবহৃত হচ্ছে।
- `Asia/Dhaka` local timezone configure করছে।

Step 2: Create `tasks.py`

```
from celery_app import app


@app.task
def add(a, b):
    return a + b


@app.task
def multiply(a, b):
    return a * b
```

Step 3: Start the Worker

Project directory থেকে:

```
celery -A celery_app worker --loglevel=INFO
```

Step 4: Execute the Task

Python shell অথবা অন্য Python file থেকে:

```
from tasks import add

result = add.delay(10, 20)

print(result.id)
print(result.get(timeout=10))
```

Expected result:

```
30
```

`result.get()` result-এর জন্য অপেক্ষা করে। Web request-এর মধ্যে দীর্ঘ সময় result-এর জন্য অপেক্ষা করলে asynchronous processing-এর সুবিধা নষ্ট হতে পারে।

---

5. Django Project Structure

একটি Django project-এর উদাহরণ:

```
myproject/
├── manage.py
├── myproject/
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   ├── asgi.py
│   └── celery.py
└── orders/
    ├── __init__.py
    ├── models.py
    ├── views.py
    └── tasks.py
```

ধরা যাক Django project-এর নাম `myproject`।

Step 1: Create `myproject/celery.py`

```
import os

from celery import Celery

os.environ.setdefault(
    "DJANGO_SETTINGS_MODULE",
    "myproject.settings",
)
```

```
app = Celery("myproject")

app.config_from_object(
    "django.conf:settings",
    namespace="CELERY",
)

app.autodiscover_tasks()
```

ব্যাখ্যা:

- `DJANGO_SETTINGS_MODULE` Django settings নির্ধারণ করে।
- `config_from_object()` Django settings থেকে Celery configuration নেয়।
- `namespace="CELERY"` থাকার কারণে settings-এর configuration variable-গুলো `CELERY_` prefix দিয়ে লিখতে হয়।
- `autodiscover_tasks()` installed Django apps-এর `tasks.py` খুঁজে বের করে।

Step 2: Update `myproject/__init__.py`

```
from .celery import app as celery_app

__all__ = ("celery_app",)
```

এটি Django project load হওয়ার সময় Celery app import করতে সাহায্য করে।

Step 3: Configure `settings.py`

```
CELERY_BROKER_URL = "redis://localhost:6379/0"

CELERY_RESULT_BACKEND = "redis://localhost:6379/1"

CELERY_ACCEPT_CONTENT = ["json"]

CELERY_TASK_SERIALIZER = "json"

CELERY_RESULT_SERIALIZER = "json"

CELERY_TIMEZONE = "Asia/Dhaka"

CELERY_ENABLE_UTC = True
```

Redis server চালু আছে এবং Django application থেকে accessible—এটি নিশ্চিত করতে হবে।

Step 4: Create `orders/tasks.py`

```
from celery import shared_task
```

```
@shared_task
def add_numbers(a, b):
    return a + b
```

`shared_task` ব্যবহার করলে Django app-এর task-গুলো project-এর Celery app-এর সঙ্গে কাজ করতে পারে, explicit app import না করেও।

Step 5: Start Celery Worker

Django project-এর `manage.py` -এর directory থেকে:

```
celery -A myproject worker --loglevel=INFO
```

Step 6: Call the Task

```
from orders.tasks import add_numbers

result = add_numbers.delay(10, 20)
```

Task worker execute করবে। Result প্রয়োজন হলে:

```
print(result.get(timeout=10))
```

---

6. Celery Task Calling Methods

`delay()`

সহজভাবে task queue-তে পাঠানোর জন্য:

```
send_email.delay("user@example.com")
```

`apply_async()`

অতিরিক্ত options দিতে:

```
send_email.apply_async(
    args=["user@example.com"],
    countdown=10,
)
```

এটি task-কে প্রায় ১০ সেকেন্ড পরে execute করার জন্য schedule করে।

`countdown`

```
send_email.apply_async(
    args=["user@example.com"],
    countdown=60,
)
```

প্রায় এক মিনিট পরে task execute করার অনুরোধ করে।

`eta`

```
from datetime import datetime, timedelta, timezone

send_email.apply_async(
    args=["user@example.com"],
    eta=datetime.now(timezone.utc) + timedelta(minutes=5),
)
```

ETA হলো নির্দিষ্ট execution time। এটি কঠোর real-time deadline নয়; worker availability এবং broker-এর অবস্থার কারণে execution দেরি হতে পারে।

দীর্ঘ সময় ভবিষ্যতের task-এর জন্য broker-এ অসংখ্য delayed message জমিয়ে না রেখে database-backed scheduling বা periodic scheduler বিবেচনা করো।

`expires`

```
send_email.apply_async(
    args=["user@example.com"],
    expires=300,
)
```

Task পাঁচ মিনিটের মধ্যে শুরু হওয়ার সুযোগ না পেলে expire হতে পারে। এটি নির্ভুলভাবে five-minute execution guarantee নয়।

`task_id`

```
send_email.apply_async(
    args=["user@example.com"],
    task_id="welcome-email-123",
)
```

নির্দিষ্ট ID ব্যবহার করা সম্ভব। তবে শুধু একই task ID ব্যবহার করলেই duplicate execution বন্ধ হয় না।

---

7. Practical Example: Send Email in Background

`settings.py`-এ email configuration করো। নিচের উদাহরণ SMTP settings-এর template:

```
EMAIL_BACKEND = "django.core.mail.backends.smtp.EmailBackend"

EMAIL_HOST = "smtp.example.com"
EMAIL_PORT = 587
EMAIL_USE_TLS = True

EMAIL_HOST_USER = "your-email@example.com"
```

```
EMAIL_HOST_PASSWORD = "load-from-environment-variable"

DEFAULT_FROM_EMAIL = EMAIL_HOST_USER
```

বাস্তব SMTP credentials environment variable থেকে load করবে; source code-এ password hardcode করবে না।

`orders/tasks.py`

```
from celery import shared_task
from django.conf import settings
from django.core.mail import send_mail


@shared_task
def send_welcome_email(email):
    send_mail(
        subject="Welcome!",
        message="Welcome to our platform.",
        from_email=settings.DEFAULT_FROM_EMAIL,
        recipient_list=[email],
        fail_silently=False,
    )

    return f"Email sent to {email}"
```

Call the Task

```
send_welcome_email.delay("user@example.com")
```

Django response email delivery-এর জন্য অপেক্ষা করবে না। তবে task queue-তে পাঠানো এবং email সফলভাবে deliver হওয়া আলাদা বিষয়।

---

8. Database Operations and Celery

ধরা যাক user তৈরি হওয়ার পরে email পাঠাতে হবে।

```
from django.contrib.auth import get_user_model
from django.db import transaction

from orders.tasks import send_welcome_email

User = get_user_model()
```

```
def register_user(email, password):
    with transaction.atomic():
        user = User.objects.create_user(
            email=email,
            password=password,
        )

        transaction.on_commit(
            lambda: send_welcome_email.delay(user.email)
        )

    return user
```

কেন `transaction.on_commit()`?

Django database transaction commit হওয়ার আগেই task worker database query চালালে worker user record খুঁজে নাও পেতে পারে।

`transaction.on_commit()` callback-টি transaction সফলভাবে commit হওয়ার পরে চালায়।

গুরুত্বপূর্ণ: database commit হওয়ার পর broker-এ task publish ব্যর্থ হলে task হারানোর সম্ভাবনা থাকতে পারে। অত্যন্ত গুরুত্বপূর্ণ business event-এর জন্য transactional outbox pattern বিবেচনা করো।

---

9. Task Retry and Error Handling

Network সমস্যা, SMTP failure বা external API error-এর কারণে task ব্যর্থ হতে পারে।

```
from celery import shared_task


@shared_task(
    bind=True,
    autoretry_for=(ConnectionError, TimeoutError),
    retry_backoff=True,
    retry_kwargs={"max_retries": 5},
)
def fetch_external_data(self, url):
    # Replace this with a real HTTP request.
    return {"url": url, "status": "processed"}
```

ব্যাখ্যা:

- `bind=True` দিলে task function প্রথম argument হিসেবে task instance পায়।
- `autoretry_for` নির্দিষ্ট exception হলে retry করতে সাহায্য করে।
- `retry_backoff=True` retry-গুলোর মাঝে ক্রমবর্ধমান বিরতি তৈরি করে।

- `max_retries=5` সর্বোচ্চ retry সংখ্যা নির্ধারণ করে।

Manual Retry

```
from celery import shared_task


@shared_task(bind=True, max_retries=3)
def process_payment(self, payment_id):
    try:
        # Perform payment-related operation.
        pass

    except ConnectionError as exc:
        raise self.retry(
            exc=exc,
            countdown=10,
        )
```

Payment processing-এর মতো sensitive কাজের ক্ষেত্রে retry-এর আগে idempotency নিশ্চিত করা জরুরি।

---

10. Task States

Celery-এর সাধারণ task states:

State	Meaning
`PENDING`	State জানা নেই বা task pending
`STARTED`	Task execution শুরু হয়েছে, যদি tracking enabled থাকে
`RETRY`	Task retry-এর জন্য

	নির্ধারিত
`SUCCESS`	Task সফল হয়েছে
`FAILURE`	Task ব্যর্থ হয়েছে
`REVOKED`	Task revoke করা হয়েছে

Check Result

```
result = add_numbers.delay(5, 10)

print(result.id)
print(result.state)
```

`STARTED` tracking default configuration-এ সবসময় চালু থাকে না। Result backend না থাকলে task state এবং return value নির্ভরযোগ্যভাবে query করার সুবিধাও সীমিত হয়।

Get Result

```
result.get(timeout=10)
```

Production web endpoint-এ দীর্ঘ সময় blocking করে result পাওয়ার পরিবর্তে task ID return করে আলাদা status endpoint রাখা ভালো।

---

11. Task Time Limits

একটি task যেন অনির্দিষ্টকাল ধরে worker দখল করে না রাখে, তার জন্য time limit ব্যবহার করা যায়।

```
from celery import shared_task


@shared_task(
    soft_time_limit=50,
    time_limit=60,
)
def process_large_file(file_id):
    # Process file.
    return "Completed"
```

- `soft_time_limit` : সময়সীমা পার হলে task-কে soft-limit exception দিয়ে থামানোর সুযোগ দেয়।
- `time_limit` : hard limit; worker task execution বন্ধ করতে পারে।

সব operating system ও worker pool-এ hard time limit একইভাবে কাজ করে না। প্রয়োজন অনুযায়ী পরীক্ষা করতে হবে।

---

12. Task Routing and Queues

বিভিন্ন ধরনের task আলাদা queue-তে পাঠানো যায়।

`settings.py`

```
CELERY_TASK_ROUTES = {
    "orders.tasks.send_welcome_email": {
        "queue": "emails",
    },
    "orders.tasks.generate_report": {
        "queue": "reports",
    },
}
```

Start Email Worker

```
celery -A myproject worker \
  --loglevel=INFO \
  -Q emails
```

Start Report Worker

```
celery -A myproject worker \
  --loglevel=INFO \
  -Q reports
```

এভাবে report processing-এর ভারী কাজ email task-গুলোর worker capacity দখল করে নেওয়ার ঝুঁকি কমানো যায়।

---

13. Periodic Tasks with Celery Beat

Celery Beat নির্দিষ্ট schedule অনুযায়ী task publish করে। Worker task execute করে।

Install

```
pip install celery
```

Configure Schedule

```
from celery.schedules import crontab

CELERY_BEAT_SCHEDULE = {
    "daily-report": {
        "task": "orders.tasks.generate_daily_report",
        "schedule": crontab(hour=2, minute=0),
    },
}
```

Create Task

```
from celery import shared_task


@shared_task
def generate_daily_report():
    # Generate and save report.
    return "Daily report generated"
```

Run Worker

```
celery -A myproject worker --loglevel=INFO
```

Run Beat Scheduler

```
celery -A myproject beat --loglevel=INFO
```

এই schedule সাধারণত configured timezone অনুযায়ী প্রতিদিন রাত ২টায় task publish করার জন্য তৈরি।

সতর্কতা: একই schedule-এর একাধিক Beat instance চালালে duplicate task publish হতে পারে। সাধারণত একটি scheduler instance চালাও, অথবা ব্যবহৃত scheduler-এর distributed coordination নিশ্চিত করো।

---

14. Cron-Style Scheduling

প্রতিদিন:

```
crontab(hour=0, minute=0)
```

প্রতি ঘণ্টায়:

```
crontab(minute=0)
```

প্রতি ১৫ মিনিটে:

```
crontab(minute="*/15")
```

প্রতি সোমবার সকাল ৯টায়:

```
crontab(
    hour=9,
    minute=0,
    day_of_week=1,
)
```

সোমবার থেকে রবিবার পর্যন্ত সপ্তাহের দিন নির্ধারণের জন্য Celery cron syntax ব্যবহার করে। Timezone এবং daylight-saving আচরণ deployment environment-এ পরীক্ষা করা উচিত।

---

15. Celery Canvas: Chain, Group, Chord

একাধিক task-এর workflow তৈরি করতে Celery Canvas ব্যবহার করা যায়।

Chain

একটি task শেষ হলে তার result পরবর্তী task-এ যায়।

```
from celery import chain

workflow = chain(
    add_numbers.s(10, 20),
    multiply.s(2),
)

result = workflow.apply_async()
```

প্রথম task-এর result `30`; দ্বিতীয় task-এ সেটি প্রথম argument হিসেবে যাবে। অর্থাৎ `multiply(30, 2)`।

Group

একাধিক independent task একসঙ্গে dispatch করা:

```
from celery import group

job = group(
    add_numbers.s(1, 2),
    add_numbers.s(3, 4),
    add_numbers.s(5, 6),
)

result = job.apply_async()
```

Chord

একটি group শেষ হওয়ার পরে callback execute করা:

```
from celery import chord, group


def combine_results(results):
    return sum(results)


workflow = chord(
    group(
        add_numbers.s(1, 2),
        add_numbers.s(3, 4),
    ),
    combine_results.s(),
)

result = workflow.apply_async()
```

বাস্তব project-এ chord callback এবং group result coordination-এর জন্য উপযুক্ত result backend প্রয়োজন হতে পারে। Backend ও broker-এর documentation দেখে compatibility নিশ্চিত করো।

---

16. Task Idempotency

Idempotent task হলো এমন task যেটি একই input নিয়ে একাধিকবার execute হলেও final business result অনাকাঙ্ক্ষিতভাবে পরিবর্তিত হয় না।

Unsafe Example

```
@shared_task
def add_balance(user_id, amount):
    user = User.objects.get(id=user_id)
    user.balance += amount
    user.save()
```

Task দুইবার execute হলে balance-ও দুইবার বাড়তে পারে।

Better Approach

Business requirement অনুযায়ী unique transaction ID ব্যবহার করে database-এ transaction record তৈরি করো এবং একই transaction দ্বিতীয়বার process করা আটকাও।

```
@shared_task
def process_payment(payment_id):
    payment = Payment.objects.get(id=payment_id)

    if payment.status == "completed":
```

```
            return "Already processed"

        # Perform a properly idempotent payment workflow.
```

শুধু status পরীক্ষা করাই concurrent execution-এর জন্য যথেষ্ট নয়। Database constraints, locking বা atomic update দিয়ে race condition প্রতিরোধ করতে হবে।

মনে রাখবে: Celery task একাধিকবার execute হতে পারে। Retry, worker crash এবং acknowledgement behavior বিবেচনায় task-কে duplicate-safe করে ডিজাইন করা গুরুত্বপূর্ণ।

---

17. Worker Concurrency

Worker একসঙ্গে কতগুলো task execute করবে, সেটি concurrency দিয়ে নিয়ন্ত্রণ করা যায়।

```
celery -A myproject worker --concurrency=4
```

এখানে চারটি worker process/slot ব্যবহারের অনুরোধ করা হচ্ছে, worker pool-এর ধরন অনুযায়ী।

Pool Options

- `prefork`: process-based execution; সাধারণত default।
- `threads`: thread-based execution।
- `solo`: একক execution; debugging-এ উপকারী।
- `gevent` / `eventlet`: বিশেষ asynchronous I/O use case-এর জন্য; compatibility যাচাই করতে হয়।

Windows-এ Celery-এর কিছু pool ও multiprocessing behavior Linux-এর মতো নয়। Development-এ প্রয়োজন হলে `--pool=solo` ব্যবহার করা যায়:

```
celery -A myproject worker --pool=solo --loglevel=INFO
```

Production deployment-এর জন্য সাধারণত Linux environment ব্যবহার করা হয়।

---

18. Task Acknowledgement and Reliability

Celery task delivery এবং execution-এর মধ্যে পার্থক্য বোঝা জরুরি।

- **Acknowledgement:** worker broker-কে জানায় যে message গ্রহণ বা process করা হয়েছে।
- **Retry:** ব্যর্থ task আবার চালানোর ব্যবস্থা।
- **Late acknowledgement:** task শেষ হওয়ার পরে acknowledgement দেওয়া।
- **Worker loss:** worker crash করলে task পুনরায় delivery হবে কি না, তা configuration এবং broker behavior-এর ওপর নির্ভর করে।

Late acknowledgement চালু করার উদাহরণ:

```
@shared_task(
    acks_late=True,
```

```
    reject_on_worker_lost=True,
)
def process_order(order_id):
    # Perform idempotent processing.
    return "Processed"
```

এগুলো সব task-এর জন্য অন্ধভাবে enable করবে না। Worker crash বা repeated failure-এর ক্ষেত্রে task পুনরায় execute হতে পারে। Duplicate-safe business logic, retry policy এবং monitoring প্রয়োজন।

---

19. Celery Monitoring

Inspect Active Tasks

```
celery -A myproject inspect active
```

Inspect Scheduled Tasks

```
celery -A myproject inspect scheduled
```

Inspect Registered Tasks

```
celery -A myproject inspect registered
```

Inspect Worker Statistics

```
celery -A myproject inspect stats
```

List Workers

```
celery -A myproject status
```

Flower Dashboard

Flower হলো Celery monitoring-এর একটি পরিচিত web tool।

Install:

```
pip install flower
```

Run:

```
celery -A myproject flower
```

Dashboard সাধারণত local port `5555` -এ পাওয়া যায়। Production-এ authentication, network restrictions এবং HTTPS-এর মতো access controls ছাড়া monitoring dashboard public করে দেবে না।

---

20. Django Cache and Celery

Django cache এবং Celery এক জিনিস নয়।

- Django cache: দ্রুত data retrieve করার জন্য।
- Celery: background-এ কাজ execute করার জন্য।
- Redis: দুটিরই backend হিসেবে ব্যবহার করা যেতে পারে, তবে প্রয়োজন অনুযায়ী আলাদা Redis instance বা logical isolation ভালো হতে পারে।

উদাহরণ:

```
from django.core.cache import cache


def get_product(product_id):
    key = f"product:{product_id}"

    product = cache.get(key)

    if product is None:
        product = Product.objects.get(id=product_id)
        cache.set(key, product, timeout=300)

    return product
```

Background task:

```
@shared_task
def refresh_product_cache(product_id):
    key = f"product:{product_id}"
    cache.delete(key)

    return "Cache invalidated"
```

একই Redis ব্যবহার করলে memory limit, eviction policy, key namespace এবং availability requirements বিবেচনা করো।

---

21. Django REST Framework API Example

একটি API request-এর মাধ্যমে report generation task queue-তে পাঠানো যায়।

`reports/tasks.py`

```
from celery import shared_task


```

```
@shared_task
def generate_report(report_id):
    # Generate report and save its status/result.
    return {
        "report_id": report_id,
        "status": "completed",
    }
```

`reports/views.py`

```
from rest_framework.response import Response
from rest_framework.views import APIView

from reports.tasks import generate_report


class GenerateReportView(APIView):
    def post(self, request):
        report_id = 123  # Replace with a real persisted report ID.

        task = generate_report.delay(report_id)

        return Response(
            {
                "message": "Report queued",
                "task_id": task.id,
            },
            status=202,
        )
```

`202 Accepted` বোঝায় request গ্রহণ করা হয়েছে; report সম্পন্ন হয়েছে—এমন নিশ্চয়তা নয়।

Production Pattern

1. Database-এ report record তৈরি করো।
2. Record-এর status `pending` রাখো।
3. Celery task queue-তে পাঠাও।
4. Worker report তৈরি করবে।
5. Database-এ status `completed` অথবা `failed` করবে।
6. Client একটি status endpoint দিয়ে progress দেখবে।

শুধু Celery result backend-এর ওপর নির্ভর না করে business-critical status database-এ রাখলে API design এবং audit করা সহজ হয়।

---

22. Common Errors and Solutions

Error: Connection refused

সম্ভাব্য কারণ:

- Redis চালু নেই।
- ভুল hostname বা port।
- Docker network configuration ভুল।

সমাধান:

```
redis-cli ping
```

Docker:

```
docker ps
```

Error: Received unregistered task

সম্ভাব্য কারণ:

- Task module discover হয়নি।
- Task name ভুল।
- Worker পুরোনো code নিয়ে চলছে।

সমাধান:

- `tasks.py` সঠিক Django app-এ রাখো।
- App-টি `INSTALLED_APPS` -এ আছে নিশ্চিত করো।
- `autodiscover_tasks()` configure করো।
- Worker restart করো।

Error: Task executes synchronously

সম্ভাব্য কারণ:

- `.delay()` বা `.apply_async()` -এর পরিবর্তে function সরাসরি call করা হয়েছে।
- `task_always_eager=True` চালু আছে।
- ভুল environment বা configuration ব্যবহার হচ্ছে।

Error: Task result is missing

সম্ভাব্য কারণ:

- Result backend configure করা হয়নি।
- Task result expire হয়েছে।
- Task এখনো শেষ হয়নি।
- Result backend-এ connection সমস্যা আছে।

Error: Worker memory usage keeps growing

সম্ভাব্য কারণ:

- বড় data memory-তে load করা।
- memory leak।
- বড় task result।
- অতিরিক্ত concurrency।

সমাধান:

- বড় data chunk করে process করো।
- Task input হিসেবে প্রয়োজনীয় ID পাঠাও।
- Worker memory monitor করো।
- প্রয়োজনে worker recycling configuration বিবেচনা করো।

---

23. Production Best Practices

1. Broker ও result backend-এর configuration environment variable থেকে নাও।
2. Task-এ বড় object বা model instance না পাঠিয়ে ID পাঠাও।
3. Task idempotent করে তৈরি করো।
4. Retry সীমিত রাখো এবং backoff ব্যবহার করো।
5. External API request-এ timeout নির্ধারণ করো।
6. Worker, Beat এবং web application আলাদাভাবে deploy করো।
7. গুরুত্বপূর্ণ task-এর business status database-এ রাখো।
8. Broker, queue length, task failure এবং worker health monitor করো।
9. Worker concurrency CPU, memory এবং workload অনুযায়ী tune করো।
10. Email, report এবং long-running task আলাদা queue-তে route করতে পারো।
11. Redis-এ authentication, private networking এবং access control ব্যবহার করো।
12. Task arguments ও logs-এ password, token বা sensitive data রেখো না।
13. Celery, Django, Redis এবং Python-এর compatible versions pin করো।
14. Graceful shutdown, worker restart এবং deployment-এর সময় in-flight task-এর behavior পরীক্ষা করো।
15. At-least-once execution এবং duplicate delivery-এর সম্ভাবনা মাথায় রেখে design করো।

---

24. Docker Compose Example

একটি development setup-এ Django, Redis এবং Celery worker চালানোর জন্য `compose.yaml`-এর উদাহরণ:

```
services:
  redis:
    image: redis:7
    command: redis-server
    ports:
      - "127.0.0.1:6379:6379"

  web:
    build: .
    command: python manage.py runserver 0.0.0.0:8000
    volumes:
      - .:/app
    working_dir: /app
    ports:
      - "8000:8000"
    depends_on:
      - redis

  worker:
    build: .
    command: celery -A myproject worker --loglevel=INFO
    volumes:
      - .:/app
    working_dir: /app
    depends_on:
      - redis
```

এই উদাহরণে ধরে নেওয়া হয়েছে যে project-এর জন্য উপযুক্ত `Dockerfile` আছে এবং Django dependencies install করা হয়েছে।

Docker network-এ Django ও worker-এর Redis URL হবে:

```
CELERY_BROKER_URL = "redis://redis:6379/0"
CELERY_RESULT_BACKEND = "redis://redis:6379/1"
```

এখানে `localhost` ব্যবহার করা যাবে না, কারণ container-এর ভেতরে `localhost` বলতে সেই container-কে বোঝায়।

`depends_on` Redis process-কে ready হওয়া পর্যন্ত অপেক্ষা করায় না। Production-এ health check, connection retry, secure credentials, persistence এবং resource limits configure করো। Development example-এর `runserver` production server হিসেবে ব্যবহার করবে না।

---

25. Security Notes

- Broker-এ access সীমাবদ্ধ রাখো।
- Redis public internet-এ expose করবে না।
- Broker URL ও credentials environment variable বা secret manager-এ রাখো।
- `pickle` serializer-এর মতো unsafe serialization format অপ্রয়োজনে ব্যবহার করবে না।
- JSON serializer দিয়ে সাধারণ primitive data পাঠানো অধিক নিরাপদ default।
- Task arguments validate করো।
- Task execution-এর আগে authorization-sensitive operation যাচাই করো।
- Logs-এ sensitive information প্রকাশ করো না।
- Monitoring dashboard নিরাপদ করো।

Celery queue-তে message পাঠাতে পারা মানেই worker-এ code execution করার সুযোগ দেওয়া হচ্ছে—তাই broker access খুবই গুরুত্বপূর্ণ।

---

26. Interview Questions and Answers

Q1. What is Celery?

Celery হলো Python distributed task queue system, যা background এবং asynchronous task execute করতে ব্যবহৃত হয়।

Q2. Why use Celery with Django?

Email sending, report generation, image processing ও external API calls-এর মতো দীর্ঘ কাজ web request থেকে আলাদা রাখতে।

Q3. What is a broker?

Broker task message গ্রহণ করে queue করে এবং worker-এর কাছে পৌঁছে দেয়। উদাহরণ: Redis, RabbitMQ।

Q4. What is a worker?

Worker queue থেকে task নিয়ে execute করে।

Q5. What is Celery Beat?

Celery Beat নির্দিষ্ট সময়সূচি অনুযায়ী periodic task publish করে।

Q6. What is a result backend?

Task-এর result ও state সংরক্ষণ করে, যদি configure করা থাকে।

Q7. What is the difference between `delay()` and `apply_async()`?

`delay()` সহজ syntax; `apply_async()` countdown, ETA, queue এবং অন্যান্য execution options দেওয়ার সুবিধা দেয়।

Q8. What is a bound task?

`bind=True` দিয়ে তৈরি task, যা প্রথম argument হিসেবে নিজের task instance পায়।

Q9. Why use `transaction.on_commit()`?

Database transaction commit হওয়ার আগে worker task execute করে যেন অসম্পূর্ণ database state না দেখে, সেই ঝুঁকি কমাতে।

Q10. What is task idempotency?

একই operation একাধিকবার execute হলেও অনাকাঙ্ক্ষিত duplicate side effect এড়ানোর বৈশিষ্ট্য।

Q11. What is task retry?

Task নির্দিষ্ট error-এর কারণে ব্যর্থ হলে আবার execute করার ব্যবস্থা।

Q12. What is task routing?

Task-কে নির্দিষ্ট queue-তে পাঠানোর ব্যবস্থা।

Q13. What is the difference between Celery and Django signals?

Django signals application event-এর প্রতিক্রিয়ায় code execute করে। Celery background worker-এ task execute করে। Signal handler থেকে Celery task queue-তে পাঠানো যায়, তবে signal নিজে background queue নয়।

Q14. Can Celery guarantee exactly-once execution?

সাধারণভাবে exactly-once execution ধরে নেওয়া নিরাপদ নয়। Duplicate delivery ও retry হতে পারে; idempotent task এবং database constraints দিয়ে business correctness নিশ্চিত করতে হয়।

Q15. Can Redis be both broker and result backend?

হ্যাঁ। আলাদা Redis database index বা পৃথক instance ব্যবহার করা যায়; তবে data retention, memory, eviction এবং reliability requirements বিবেচনা করতে হবে।

Q16. What happens if a worker crashes?

আচরণ acknowledgement mode, broker, worker pool এবং task configuration-এর ওপর নির্ভর করে। কিছু task redeliver হতে পারে, অন্য কিছু হারানোর ঝুঁকি থাকতে পারে। তাই retry ও idempotency জরুরি।

Q17. How do you monitor Celery?

Worker inspection commands, logs, metrics এবং Flower-এর মতো monitoring tool দিয়ে।

Q18. How do you schedule a task every day?

Celery Beat-এর সঙ্গে `crontab(hour=..., minute=...)` schedule ব্যবহার করে।

---

27. Practice Projects

Beginner

- Addition task
- Background email sender

- Task status checker

Intermediate

- Django registration email system
- PDF report generator
- Image processing pipeline
- Failed task retry system

Advanced

- Scheduled daily report service
- Separate email and report queues
- Redis-backed Celery system with monitoring
- Idempotent payment-processing workflow
- Reliable notification pipeline with database status
- Transactional outbox implementation

---

28. Recommended Learning Roadmap

1. Python functions ও exceptions
2. Django views, models ও database transactions
3. Redis basics
4. Celery app, broker ও worker
5. Task calling: `.delay()` এবং `.apply_async()`
6. Django integration
7. Email এবং report generation
8. Retry, error handling ও idempotency
9. Celery Beat ও periodic tasks
10. Queues, routing ও concurrency
11. Monitoring, reliability ও security
12. Docker-based development
13. Production deployment ও troubleshooting

Final Summary

- **Django** — web application ও API পরিচালনা করে।
- **Celery** — background task execute করে।
- **Redis** — broker বা result backend হিসেবে ব্যবহার করা যায়।

- **Celery Worker** — task execute করে।
- **Celery Beat** — periodic task schedule করে।
- `delay()` — task queue-তে পাঠানোর সহজ উপায়।
- `apply_async()` — অতিরিক্ত execution options দেয়।
- `transaction.on_commit()` — database commit-এর পরে task publish করতে সাহায্য করে।
- **Idempotency** — duplicate execution থেকে business side effect রক্ষা করতে সাহায্য করে।

Practical goal: Django + Celery + Redis ব্যবহার করে একটি background email system তৈরি করো, তারপর retry, task status, periodic scheduling এবং Docker integration যোগ করো।